Project pack for Apeksha — give this whole file to your AI

To the AI reading this: you are helping Apeksha, a member of a four-person university team, finish a school portal (vanilla PHP 8, HTML, CSS, JS; no frameworks). This pack has five parts, in this order:

1. **The rules** — every line of code must follow them.
2. **How it works** — the folders, one request through every file, security, white-label, the database.
3. **Why** — the reason behind each rule.
4. **Apeksha's tasks** — what to build, in order, and the pages and files involved.
5. **The decided details** — every rule already agreed for the tasks left.

You cannot see the code. Ask Apeksha to paste any file you need (all code is in the `code_lecole/` folder) — never guess what a file contains. Do not write code until Apeksha asks for it; plan first. Give whole files, not fragments.

Part 1 — The rules

Claude Code reads this file on every session in this folder. Everyone on the team follows the same rules, so the code stays as simple and consistent as it is now. Start Claude Code in `code_lecole/` (this folder); the team's documents are in `../team/`.

The project

- A school portal: five portals (student, parent, teacher, management, admin) + the public landing page.
- Vanilla PHP 8 / HTML / CSS / JS. No frameworks or libraries unless the lead approves.
- Each page's **look is finished**. Backend work keeps the page looking exactly as it does now.
- The code must stay simple enough to explain line by line in the viva and to change live in the code check.

Tasks

- Each person's tasks: `../team/handover/{name}.md`. Work only inside your own task unless the lead says so.
- Branch from `dev`, open the pull request into `dev`. The lead (repo owner) reviews and merges every pull request, and alone merges `dev` into `main`. Nothing reaches `dev` or `main` without that review.

How we work

- One task = one branch = one pull request (`../team/GIT.md`). Small commits, one change each.
- Before building, read: the page, its components, CSS and JS — and the backend under it (route, controller, model, config) just as closely. Then read the task in `../team/CHECKLIST.md` ("Backends left to connect") and your handover file. Use `/next-feature <task>` for the plan; build only after the lead approves the plan.
- "Don't build yet" / "discuss" = no file changes.
- Track work in `../team/CHECKLIST.md`: tick items when done and verified; add new findings there.
- Verify every change: `php -l` on PHP, `bash bin/check.sh` (0 errors), open the page in the browser (http://localhost:8080) and compare it with how it looked before. For anything that takes a form or a background

request, also attack it with curl: no CSRF token, `field[]=x`, the wrong role, signed out. Reading code is not testing it.
- Report honestly: what works, what is still example data (mock), what was not checked.

Code rules

- Views in **per-role folders** (`app/Views/student/dashboard.php` …). Shared pieces are components in `app/Views/components/`; per-use differences come from a `$variant` (multi-variant components), not copied files.
- One component = its PHP + its own CSS file + its own JS file. `global.css` only holds tokens, resets and the app shell.
- Page rows (how components line up together) live in `public/assets/css/layout/{row}-grid.css` with `l-…-grid` classes, e.g. `l-chart-calendar-grid`; a page lists its rows in `$grids`. Only arranging code goes there (grid, columns, gaps, screen sizes, space between rows) — never a component's own look.
- One name per colour (`global.css` tokens). No duplicate rules, no `!important`, no inline `style=""`, no dead code. Dead code includes: a file no page loads, a parameter nobody passes, a setting / colour token / route nobody uses. One rule in one place: role lists and "who may do what" live in `config/permissions.php` (or the feature's config).
- Models are the only place with SQL (prepared statements). Controllers decide. Views only display (`e()` on every value). SQL built from array keys only from a fixed list in the code. Several changes that belong together → `transaction()`.
- Form values only through `postText()` / `postPassword()` / `postList()`, never `$_POST` directly.
- CSRF is checked once, by `core/Router.php`, for every POST — never by hand. Logout is POST. A background (JSON) request gets JSON back on every path (`refuse()`); a crash shows `errors/error.php`, never code or file paths.
- Never reveal which accounts exist: an unknown ID and a real one get the same message. Anything that sends an email or a code has a wait between sends. Secrets (passwords, keys) live in `.env` only, never in code or in git.
- White-label: another school must work by changing `config/white_label/` only. Name, logo, contacts → `school()`; Grade / Class / Term / Student… → `word()` / `plural()`; modules → `feature()`; dates → `country('date_format')` (PHP) and `<html lang>` (JS); "today" = PHP's date. Never type the school's name, domain, phone code or words — not even in comments, JS names or storage keys. Every setting a feature needs is read from there.
- A school rule with a number in it (a pass mark, an age limit, a maximum) goes in `config/white_label/rules.php` (read with `rule('…')`) or the feature's config — never typed into a controller.
- Names match the screen: the words the page shows are the words in tables, models, methods and config keys (the page says "Audit Log" → the table is `audit_log`). Database changes are **updates**: `database/updates/NNN_name.sql`, run with `php database/update.php` — never edit an update that is already in `dev`.
- Formatting: PHP and JS `{` on the next line; CSS `selector {` on the same line, one property per line.

Explaining

Short, line by line, no long paragraphs. Show which file calls which and what is passed. Give complete code. The person asking must be able to explain every line themselves: if they can't, explain it again (`/explain`).

Running

- `README.md` : Docker MySQL on port 3308 (root / root, database `l_ecole`), phpMyAdmin http://localhost:8086, the site on http://localhost:8080. Demo accounts' password: `Password@123`.

Part 2 — How it works

1. One request, start to finish

```
browser → public/index.php → core/bootstrap.php → Core\App → routes/web.php → Core\Router → a controller
→ a view
```

Step	File	What it does
1	`public/index.php`	The only PHP file the browser can reach. A real file (CSS, JS, image) is sent as it is.
2	`core/bootstrap.php`	Reads `.env`, sets the timezone (`country('timezone')`), starts the session (cookie: `httponly`, `SameSite=Lax`, strict mode), and decides how errors show (`APP_DEBUG`).
3	`core/App.php`	Loads `routes/web.php`, cleans the address (`/student/` → `/student`), hands it to the Router.
4	`routes/web.php`	The list of addresses: `$router->get('/student/academic', [AcademicController::class, 'student'])`. A module switched off in `features.php` has no routes at all.
5	`core/Router.php`	Finds the address (unknown → `errors/error.php`, 404). **Every POST passes `checkCsrf()` here first**: the one place CSRF is checked.
6	`app/Controllers/…`	Decides: `requireRole('student')`, reads the form (`postText()` …), asks the models, picks the view.
7	`app/Models/…`	The only place with SQL (prepared statements, `:name` placeholders).
8	`app/Views/{role}/…`	Only shows. Every value goes through `e()`.

2. Folders

Folder	Holds
`core/`	The small framework: App, Router, Controller (base), Model (base: `transaction()`), Database (PDO), Mailer, Upload, Validator, `helpers.php`
`app/Controllers/`	One per feature (Notice, Calendar, People, Academic…)
`app/Models/`	SQL, one per table group (`UserModel`, `NoticeModel`, `Membership` …)
`app/Services/`	Work that is neither SQL nor a page: shaping data (`RecordBook`, `AchievementTimeline`, `Programmes`), the PDF (`CertificatePdf`)
`app/Mock/`	Example data for the pages whose backend isn't built yet
`app/Views/{role}/`	One page file per role and page (`student/dashboard.php`)
`app/Views/components/`	Shared pieces (`_dropdown.php`, `_pop_up.php` …); differences come from a `$variant`, not copies
`app/Views/layouts/`	`page_top.php` (head, sidebar) and `page_bottom.php` (scripts every page needs)
`config/`	Settings and rules: `permissions.php` (who may do what), one file per feature
`config/white_label/`	Everything another school changes: `school.php`, `wording.php`, `features.php`, `country.php`, `academic.php`

Folder	Holds
`public/assets/css/`	`global.css` (colour tokens, resets, the app shell), `components/` (one file per component), `layout/` (page rows: `l-…-grid`)
`public/assets/js/components/`	One file per component
`database/`	`schema.sql`, `seed.sql`, `updates/` (one numbered `.sql` file per change; `update.php` runs the new ones and remembers them in the `database_updates` table)
`bin/check.sh`	The rule check (0 errors before a hand-in)

3. How a page is made

A view sets four lists, then includes the top and bottom of the page:

```
$tabTitle = 'Notices';
$styles   = ['notice-card', 'dropdown'];       // public/assets/css/components/{name}.css
$scripts  = ['dropdown'];                      // public/assets/js/components/{name}.js
$grids    = ['notice'];                        // public/assets/css/layout/{name}-grid.css
require dirname(__DIR__) . '/layouts/page_top.php';
…  the components, e.g.  require dirname(__DIR__) . '/components/_notice_card.php';
require dirname(__DIR__) . '/layouts/page_bottom.php';
```

- A page loads only the CSS and JS it lists, so nothing unused is downloaded.
- Component = its PHP + its own CSS + its own JS. `$variant` gives the per-use differences.
- Page rows (`l-chart-calendar-grid`) only arrange: columns, gaps, screen sizes. Never a component's look.
- `Controller::page()` gives every page: `currentRole`, `currentPath`, `userName`, `csrfField`, `sessionTimeout`, and for a parent `children` + `chosenChild`.

4. Security, rule by rule

Threat	What stops it	Where
SQL injection	Prepared statements only; a column name built from array keys only from a fixed list in the code	`app/Models/`
XSS (script in a name)	`e()` on every value shown	`core/helpers.php`, every view
CSRF (another site sends our form)	A secret token in every form, checked once for every POST	`core/Router.php` → `Controller::checkCsrf()`
Wrong role	`requireRole('admin')` at the top of every action; who may do what lives in `config/permissions.php`, read with `can()`	`core/Controller.php`
Finding which accounts exist	An unknown ID, an account not activated and a wrong password get the same message; "forgot password" says the same for every ID	`AuthController`
Password guessing	Failed tries are counted; the account locks for a while (`app.login`)	`UserModel::recordFailedLogin()`
Email / code flooding	A wait between codes (`secondsUntilResend()`)	`VerificationCode`

Threat	What stops it	Where
Stolen session	`session_regenerate_id()` at sign-in, cookie `httponly` + `SameSite`, signed out after idle minutes (a warning a minute before)	`AuthController`, `bootstrap.php`, `session-timeout.js`
Strange form values (`field[]=x`)	Values only through `postText()` / `postPassword()` / `postList()` / `queryText()`: anything not plain text becomes `''`	`core/Controller.php`
Passwords	Stored with `password_hash()`, checked with `password_verify()`	`UserModel`
Error details	`APP_DEBUG=false` → people see `errors/error.php`; the details go to `storage/logs/`	`bootstrap.php`
Background (JSON) requests	Every path answers JSON, also "signed out" and "not allowed" (`refuse()`)	`core/Controller.php`
Half-saved data	Changes that belong together run in `transaction()`: all or nothing	`core/Model.php`
Secrets	Only in `.env` (never in the code, never sent to the browser)	`.env`

5. White-label: another school changes `config/white_label/` only

Need	Helper	File
Name, logo, email, phone	`school('name')`	`school.php`
Grade / Class / Term / Student…	`word('class')`, `plural('class')`, `words('{Class} {Teacher}')`	`wording.php`
Switch a module off	`feature('notices')` (its routes, sidebar link and pages go)	`features.php`
Date format, timezone	`country('date_format')`; JS reads `<html lang>`	`country.php`
Grades, ID formats	`config('white_label.academic…')`	`academic.php`

6. Features that run in the browser

File	Does
`pop-up.js`	Opens / closes `<dialog>` pop-ups (`data-pop-up-open="id"`); the browser gives Esc, the dark backdrop and focus
`question-pop-up.js`	"Are you sure?" questions (`askQuestion()`), also on forms with `data-question-*`
`unsaved-changes.js`	Asks before leaving a changed form (`data-warn-unsaved`) or closing a typed-in pop-up
`send-once.js`	Turns a form's buttons off while it sends, so a double click can't send it twice (also on sign-in / sign-up)
`session-timeout.js`	"Still There?" a minute before the idle sign-out
`feedback-banner.js`	The sliding banner (section 8); on example-data pages it shows what would happen
`dropdown.js`	Every styled dropdown (keyboard, multi-select, kept inside the window); a must-choose one (`$required`) stops its form while empty

File	Does
`sidebar.js`	Collapses the menu (remembered in a cookie); on a phone it starts collapsed

7. Example data (mock)

Pages whose backend isn't built yet (feedback, complaints, certificates, extracurricular, achievements, term marks, dashboards' numbers) read `app/Mock/*.php`. Their forms carry `data-mock-message`: nothing is saved, the banner says what would happen. Each one's real backend is listed in `team/CHECKLIST.md`.

8. Telling the person — which tool when

One question decides it: **must the person decide something, or are we just telling them?**

The situation	Tool	How
Must **decide** — it can't be undone	Question pop-up	`askQuestion({…})` or a form with `data-question-title/-text/-button`
Must **fill in or look at** something, without leaving the page	Pop-up with content	`components/_pop_up.php`
"It worked"	Sliding banner, green, leaves after a few seconds	PHP: `$this->setFlash('notice', 'Notice pinned.')` before the redirect · JS: `showFeedbackBanner('…')`
"It didn't work" after a one-click action	Sliding banner, red, **stays until ×**	PHP: `refuse()` (it sets the 'error' flash) · JS: `showFeedbackBanner('…', 'error')`
"It didn't work" in a form	Red box inside the form	the controller reads `getFlash('error')` for its form → `.c-banner--error`
A hint or warning to read while working	Box in the page	`.c-banner--info` / `--warning`

How the flash reaches the banner: `Controller::page()` reads what is left of 'notice' / 'error' → `layouts/page_bottom.php` draws the one sliding banner of the page. A form page reads its 'error' first (while its data is built), so a form's problem is shown once, inside the form. `bin/check.sh` refuses a green box in a page and a page that draws the banner itself. (The sign-in pages are outside the signed-in frame: their messages are about the form you're about to fill, so they stay boxes there.)

9. The database in three layers

Layer	What	Tables
Structure — set up once, rarely changes	the school itself	`grades`, `classes`, `stages`, `stage_subjects`, `extracurriculars`
Assignments — who is where, per school year	changes every year; earlier years are kept	`class_students`, `class_teachers`, `subject_teachers`, `teachers_in_charge` (each row has a `year`) · club members: `extracurricular_members` (now) + `extracurricular_member_history` (every membership that ended)
Records — what happened	kept for good	`marks`, `term_comments`, `achievements`, `notices`, `events`, `audit_log` …

- The current school year = the newest row of `school_years`. Year-end (B-9) adds the next one; until then a new calendar year changes nothing on its own.
- Pages read this year's assignments through the views `current_class_students`, `current_class_teachers`, `current_subject_teachers`, `current_teachers_in_charge` and `current_teaching` (every class a teacher teaches). A view is a saved SELECT: `SELECT … FROM class_students WHERE year = (newest year)`.
- Only `App\Models\Assignments` writes the assignment tables (`placeStudent`, `setClassTeacher`, `setSubjectTeacher`, `setTeacherInCharge`, `hasRoom`).
- A class that had students in an earlier year can't be deleted: its records point to it (the database refuses it, and the Academic page says so first).
- Term marks: the class teacher types them (0–100 or AB) and presses Publish Term; students and parents see only published terms. Totals, averages, positions and the automatic awards are never stored: `App\Services\TermResults` works them out from the marks every time, so a correction changes everything at once. The school's rules (Commended 80 %, Top of Class, Top of Grade, each on / off) are in `config/white_label/rules.php`, read with `rule('…')`.
- A subject with marks is never deleted or renamed: taken out of a curriculum it is retired (`retired_on`), so old reports keep showing it under its old name.

Part 3 — Why

The viva asks "why". Each rule below has one reason you can say in a sentence. How it works (the files, step by step) is in `team/HOW-IT-WORKS.md`; this file is the reasons. Code paths are inside `code_lecole/`.

What the rebuild did

Every page keeps its look from the first version. Underneath, the same pages now take far less code:

	First version	Now	
Page + component PHP (views)	15,551 lines	7,822	–50 %
CSS	17,235	12,300	–29 %
JavaScript	18,918	3,104	–84 %
Example data instead of a database	most pages	6 files left (`app/Mock/`)	your tasks remove them

(Non-blank lines, counted 2026-11-01.) How:

- **Copies became components.** The same card typed in five pages is now one component with a `$variant`.
- **JavaScript that only drew HTML became PHP.** The server builds the page; JS is left for what must happen in the browser (dropdowns, pop-ups, the calendar).
- **The browser does it for free.** `<dialog>` for pop-ups (Esc, the dark backdrop, focus), `required` for empty fields, `<form method="get">` for filters — instead of hundreds of lines of JS.
- **One look per kind of thing.** One button (`c-btn` + a look), one field (`c-field__input`), one card, one banner, one set of colours, sizes, corners and shadows (`global.css` tokens). `bin/check.sh` refuses anything else.
- **Real data, one table per thing,** with every rule in one place.

The structure

Rule	Why
Views in per-role folders (`app/Views/student/…`)	Each role's pages are easy to find; a role sees only its own pages
A page only lists components (`$styles` , `$scripts` , `$grids`)	Pages stay ~30 lines; a page loads only the CSS / JS it uses
One component = its PHP + own CSS + own JS	Change a component in one place; nothing else breaks
`$variant` instead of a copied file	One fix fixes every page that uses it
Page rows in `css/layout/*-grid.css`	Arranging (columns, gaps) is kept apart from how a piece looks
`global.css` = tokens, resets, the app shell only	One name per colour: a colour change is one line
No frameworks, no libraries (only Dompdf, for the PDF)	We can explain every line; nothing hidden

MVC on the server

Rule	Why
Models are the only place with SQL	One place to check for injection; controllers stay readable
Controllers decide, views only show	Business rules are in one place, not scattered through HTML
Services (`app/Services/`) for work that is neither SQL nor a page	e.g. `TermResults` works out averages and positions; `JoinRules` decides who may join
Totals, averages, positions are worked out live, never stored	A corrected mark changes every number at once; nothing goes stale
Assignments per school year (`class_students` with a `year`)	Last year's classes, teachers and marks stay true after year-end
Accounts and records are never deleted (switched off, archived, retired)	History and old reports keep working

Security

Rule	Why (the attack it stops)
Prepared statements only; column names only from a fixed list	SQL injection
`e()` on every printed value	XSS: a script typed into a name would run in other people's browsers
CSRF token on every POST, checked once in `core/Router.php`	Another website can't send our forms with a signed-in person's session
Logout is a POST	A link or image on another site can't sign people out
`postText()` / `postPassword()` / `postList()` only	`field[]=x` or other odd values can't crash a page or slip through
`requireRole()` + `can()` on every action	Typing another role's address does nothing
The same message for an unknown and a real account	Strangers can't find out which IDs exist
A wait between emails / codes; tries counted; a pause after wrong passwords	Inbox flooding and password guessing
Passwords with `password_hash()` ; codes stored hashed	A stolen database doesn't give away passwords
`transaction()` for changes that belong together	Never half-saved (e.g. a team and its whole roster)
JSON requests get JSON on every path (`refuse()`)	The page's script always gets an answer it understands

Rule	Why (the attack it stops)
Secrets only in `.env` (not in git)	Keys can't leak through the code
Errors show `errors/error.php`; details go to `storage/logs/`	People never see code or file paths

Rules in one place

Rule	Why
Who may do what → `config/permissions.php` (`can('…')`)	Changing who may do something is one line, and you can show the examiner the whole list
School rules with numbers → `config/white_label/rules.php` (`rule('…')`)	"Pass mark 40 instead of 35" is one line. **The code check will ask this kind of thing**
A feature's own options → `config/{feature}.php`	Limits, lists, sizes for one feature sit together

White-label (another school, no code changes)

Need	How	Why
Name, logo, contacts	`school('name')`	Another school changes `config/white_label/school.php` only
Grade / Class / Term / Student	`word('class')`, `plural()`, `words('{Class} {Teacher}')`	Another school may say "Form" or "Semester"
Switch a module off	`feature('notices')`	Its pages, routes, menu link and dashboard cards all go
Dates	`country('date_format')`, `<html lang>` in JS	Each country writes dates its own way

How we keep it clean

Tool	Does
`bin/check.sh`	Syntax, banned things (`!important`, inline styles…), dead code, duplicates, typed school words — 0 errors before every merge
The edit hook (`.claude/hooks/check-edit.sh`)	Warns Claude straight after an edit that breaks a rule
`rules-reviewer`	Reviews a branch against `CLAUDE.md` before a pull request
`/next-feature`	One complete plan before any code
`/explain`	Walks you through a file until you can explain it
`team/CHECKLIST.md`	Every decision, with its date and the reason
The GitHub check	Runs `bin/check.sh` on every pull request

Part 4 — Apeksha's tasks

Read `team/ONBOARDING.md` first: it has the overview of what the project has so far. Every code path below is inside `code_lecole/`.

Each task: branch from `dev` → `/next-feature` plan → the lead approves → small commits → pull request into `dev`. The full decisions are in `team/CHECKLIST.md` → "Backends left to connect".

1. B-11 Profile details — first (the simplest)

- **What:** My Profile still shows example values for details nobody stores yet.
- **Now:** `ProfileController::profile()` reads `app/Mock/profiles.php`; the cards each role sees: `config/profile.php` (`layout`). Find which fields are example data, store them (update file + `PeopleModel` / `UserModel`), delete the mock file.

2. B-12 Smaller ones

- **Notice attachments:** the notice form already has the upload box — save the file through `core/Upload.php` (the size the form shows is `config/notices.php` → `max_file_mb`; add the allowed types there), show it on the notice card, delete it with the notice.
- **Audit Log paging:** only the newest rows are reachable — add pages (`AuditController`, `AuditLog` model; the People directory's paging is the example: `per_page`).
- **"Changed your password" email:** after `ProfileController::password()` and the reset in `AuthController`, send an email (`core/Mailer.php`, the templates in `app/Views/emails/`).

3. B-13 Settings, edited where they belong

- **What:** admin edits school settings on screen; `config/white_label/` stays the starting values; a change is saved in a `school_settings` table that overrides the file value; every change in the Audit Log.
  - a Settings page (admin, new sidebar item): **School** tab (name, logo, emails, phone, address) · **Words** tab
  - the Academic page: **School Year** section (term start + end dates per year, `school_terms`) · **Marks rules** section (Commended %, Top of Class, Top of Grade on / off)
  - the Calendar: a **Holiday** category (whole school; admin, management)
- **Each design is shown to the lead first.** Read the full B-13 entry in the checklist ("One school timeline").
- B-9 Year-end needs the School Year dates from this task.

4. B-10 Dashboards — last (it reads everyone's data)

- **What:** the 5 dashboards' number cards, bar chart and donuts from real data (B-1 … B-6).
- **Now:** `DashboardController::show()` reads `app/Mock/dashboards.php`; every block there names its `feature`. Marks and extracurricular numbers are real already (`Services/MarksDashboard`, `ExtracurricularDashboard`) — they are the examples to follow. Replace the rest block by block; delete each mock block when it's real.

Practice (before B-11)

Make one small change on a branch `practice/apeksha` (for example: 250 members per age group instead of 300 — `config/white_label/rules.php`, `age_group_limit`), open a **draft** pull request into `dev`, and ask the lead to review it. It is closed without merging — it's only to practise the whole Git flow once.

Ask the lead

Anything outside your tasks · any decision marked □ or "discuss" in the checklist · a new table or column · a change to how a page looks.

Part 5 — The decided details (team/CHECKLIST.md, "Backends left to connect": the tasks left)

☑ done · □ to do. Each task's entry, as agreed:

- □ B-3 **Achievements** — the teacher records one (student(s), level, result, evidence upload), verification (teacher in charge → management), the student's timeline + number cards, a student's "missing record" issue, the honours board picks. Has: `achievements`, `achievement_students`. Example data now: Mock/achievements
- □ B-4 **Character certificates** — built from verified records (B-2, B-3), draft preview (pending marked), management's final verification, PDF download locked until verified, leaving date (stops new entries, warns), certificate requests + missing-record requests. Has: `certificates` (old shape). Example data now: Mock/certificates
- □ B-5 **Feedback** — the class teacher writes to a student: Student / Parent / Both (student-only invisible to parents), kinds (praise / constructive / concern), one-way, read marks. Has: `feedback` (no audience yet). Example data now: Mock/feedback
- □ B-6 **Complaints** — the parent writes (recipient: class teacher / head teacher / principal, category, subject, body, child), one-way, management marks resolved with an answer. Has: `complaints`, `complaint_messages` (a thread: the proposal says one-way). Example data now: Mock/complaints
    - **Notification list** (owner, 2026-10-29: every feature adds who is told what, plus notes on how to build it; nothing is built yet — no table, no bell. B-7 is built **last**: the owner has another idea for it, to hear first):

When	Who is told
A class's term is published (B-1)	its students + their parents
A published mark is corrected (B-1)	that student + their parents
A student shows Interest (B-2)	their parents (an Approvals card too)
A parent says yes (B-2)	the teacher in charge (a Join Request card too) · the student
A parent says no / the teacher accepts or rejects (B-2)	the student + their parents
A student is removed from a programme (B-2)	the student + their parents
Academic Watch locks / unlocks (automatic or by hand) (B-2)	the student, their parents, their teachers in charge
A new programme waits for approval (B-2)	admin · approved / rejected → whoever created it
A programme is archived (B-2)	its members + their parents
Team / position changes (B-2): placed in a team (by hand or on accept), a position set / changed, taken out of a team, their team deleted	that student (+ their parents?)
An award is recorded (B-2 / B-3)	every participant + their parents
A permission request (trip / event) is sent; its reply-by date is near (B-8)	the students' parents
Feedback written (B-5) · complaint sent / resolved (B-6) · leaving date set (B-4 / B-9)	as each item says

- □ B-7 **Notifications** — in-platform only: interest → parent, parent's yes → teacher in charge, complaint → recipient, leaving date → class teacher / TICs / management, new notice … No table yet
- □ B-8 **Permission request cards** — trips / events (name, date, place, cost) → parents approve / reject with time; the teacher in charge can't take a student without a yes. No tables yet (new; not in the old design)
- □ B-9 **Year-end** — promote every active student a grade (class mapping chosen by management), Grade 13 → leaving, class teachers reassigned, history untouched. Needs B-1's `class_enrolments`
    - Owner's ideas (2026-10-29, discuss in detail at B-9): **held back** — a student with too many low marks (e.g. more than 3 subjects under the pass mark, or a very low average) is not promoted automatically: next year they stay in the same grade. Management / admin can **allow promotion** anyway (a button on the year-end page, with a reason, logged). The thresholds and the on / off switch go in `config/white_label/rules.php`. The record book then has two books for the same grade (one per year), and the last term's report shows the result ("Promoted to Grade 8" / "Repeats Grade 7"). B-1 already keys marks and books by year, not grade.
    - Owner's idea for B-2: a student whose average is very low may be blocked from joining new clubs / sports (a switch in rules.php; existing memberships stay unless the school decides otherwise).
- □ B-10 **Dashboards** — the 5 dashboards' number cards, bar chart, donuts from B-1 … B-6. Example data now: Mock/dashboards
- □ B-11 **Profile details nobody stores yet** — My Profile's example values (Mock/profiles.php)
- □ B-12 **Smaller ones** — notice attachments (upload, the form already has the box), Audit Log paging (only the newest are reachable), "Changed your password" email
- □ B-13 **Settings, edited where they belong** (owner, 2026-10-29; right after B-1; each design shown first). `config/white_label/` stays the starting values (a new install still only changes those files); an admin change is saved in a `school_settings` table that overrides the file value; `school()`, `word()`, the rules read through one function; every change in the Audit Log.
    - Settings page (admin, new sidebar item): **School** tab (name, logo, emails, phone, address) · **Words** tab (Grade, Class, Term, Student, Teacher, Subject…) · view-only: modules, sign-ups, country (installer only: switching a module off hides pages that hold data; a timezone change shifts every date)
    - Academic page: **School Year** section (term start + end dates per year: `school_terms`, made in B-1) · **Marks rules** section (Commended %, Top of Class, Top of Grade, on / off) · class size / naming
    - Calendar: a **Holiday** category (whole school; admin, management)
    - **One school timeline** (owner, 2026-10-29): each date stays in the one table that owns it (`school_terms`, `events` incl. holidays, `term_publications`, `class_students`, admission / leaving dates, `audit_log`) — never copied into one big table, never typed into files. A `SchoolTimeline` service reads the PLANNED dates together: clash warnings when adding an event ("a public holiday (Poya)", "outside {Term} 2", "exams that day"), term start / end markers on the calendar, and the dates B-9 / B-4 need. Public holidays change every year (Poya follows the moon): admin enters them once a year in the School Year section — one by one or pasted as a list (`2027-01-13 Duruthu Full Moon Poya Day`), saved as Holiday events. No online holiday service (a library, and unreliable for Poya). Nothing country-specific in the code.